

Tipos

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 6 (hasta sección 6.11 inclusive)
- Scott: Capítulo 8

Observación: La clave de lectura aquí es una lectura superficial, se trata de tener un conocimiento general del primer abordaje de los lenguajes al tema de los tipos. La parte de análisis más significativo la abordaremos cuando hablemos del sistema de tipos.

Tipos

La discusión sobre tipos es uno de los ejes centrales de los lenguajes de programación. Los tipos son otra clase de entidad, por lo que, al igual que las variables, se definen a través de un conjunto de atributos.

La mayoría de los lenguajes incluye esta noción vinculada a expresiones y objetos de dato. Las principales razones:

- ✧ Proveen un contexto implícito para muchas operaciones, de esta manera el programador no debe explicitar el contexto.
- ✧ Limitan el conjunto de operaciones que pueden realizarse en un programa semánticamente válido.

De esta manera, los tipos tratan con las componentes de un programa que son sujeto de computación, haciendo énfasis en las propiedades de los objetos de dato y las operaciones de dichos objetos.

La discusión sobre tipos incluye tiene tres grandes áreas:

- ✧ Tipos de datos.
- ✧ Encapsulamiento y abstracción.
- ✧ Sistemas de tipos.

El primer ítem está abocado a la definición de lo que es un tipo, qué tipos son considerados tipos básicos y el recuento de los diferentes constructores de tipos compuestos que brindan los lenguajes (el material de lectura de esta clase está abocado a esta parte).

Un tipo de dato define una colección de valores y un conjunto predefinido de operaciones para esos valores.

En segundo lugar, contar con constructores de encapsulamiento y abstracción le brinda al lenguaje la posibilidad de definir nuevos tipos de datos abstractos. Esto hace que el universo de tipos pueda expandirse de manera más eficiente y correcta, dejando la relevancia de los tipos como entidad más establecida.

En tercer lugar, el sistema de tipos es un conjunto de reglas que estructuran y organizan una colección de tipos. Estas reglas y su organización tienen un grado de variedad importante y es por eso que la parte más interesante de la discusión de tipos surge del análisis de esta parte. **El objetivo del sistema de tipos de un lenguaje es lograr que los programas sean tan seguros como sea posible.**

En general, el cómputo resulta de la manipulación de esos datos. Por este motivo, el trabajo que hagan los lenguajes con respecto al conjunto de tipos que le deja disponible al programador tiene un impacto en la calidad de las soluciones. Los dos primeros ítems mencionados previamente apuntan precisamente a esto, a qué facilidades va a brindar el lenguaje para proveerle tipos al programador y qué habilidades le va a brindar para que el programador pueda definir sus propios tipos según sus necesidades.

Tipos básicos o primitivos

Estos tipos son aquellos que no están definidos en términos de otros tipos. Se incorporan en el lenguaje en el momento del diseño. Algunos de estos tipos simplemente reflejan lo que se tiene como tipo en hardware, ya que su soporte está enteramente cubierto por él (tal es el caso de algunos tipos numéricos). Otros requieren de algún soporte adicional para su implementación (como por ejemplo los strings en algunos lenguajes).

Típicamente en este grupo entran los tipos numéricos, caracter y booleanos. Tengan en cuenta que este último tipo no está disponible en todos los lenguajes, aunque contar con él mejora en general la legibilidad de los programas.

Tipos compuestos

Los tipos compuestos son una forma de unir varios tipos más simples en uno nuevo utilizando un constructor de tipo. De esta manera el lenguaje provee sintaxis para el constructor y semántica a través de operaciones embebidas. Así, por ejemplo, si se considera el tipo arreglo, tenemos la sintaxis para definir un arreglo y también operaciones para acceder a una componente, conseguir una porción de arreglo, etc.

Claramente la semántica del tipo puede expandirse definiendo unidades (abstracciones a nivel sentencia o a nivel expresión, es decir, procedimientos o funciones) para tal fin.

La lista de los constructores más comunes es:

- ✧ Strings (en algunos lenguajes este constructor se provee como un tipo básico o primitivo en lugar de un tipo compuesto).
- ✧ Enumerados.
- ✧ Subrangos.
- ✧ Conjuntos.
- ✧ Registros o producto cartesiano.
- ✧ Registros o variantes o uniones.
- ✧ Arreglos o mapeos.
- ✧ Punteros o tipos recursivos.
- ✧ Archivos.

Para cada uno de ellos hay consideraciones de diseño a tener en cuenta.

Tipos definidos por el usuario

Si bien los tipos definidos a través de los constructores de tipo mencionados en la sección anterior son tipos definidos por el programador, la habilidad del programador para definir realmente tipos complejos aparece cuando el lenguaje provee herramientas de encapsulamiento y abstracción.

Al contar con este tipo de constructores, el programador estaría en condiciones de definir TDAs y por lo tanto es responsable de determinar la estructura del tipo (conjunto de valores que lo forman) y el conjunto de operaciones con los que va a equipar al tipo. Profundizaremos en este punto más adelante.